

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week4 – Friday	Batch	23CSBTB47B
Name	Aelishala Manoj	Hall ticket No	2303A54054
Assignment Number: 7.5			
Q.No.	Question		
1	Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs Lab Objectives: - To identify and correct syntax, logic, and runtime errors in Python programs using AI tools. - To understand common programming bugs and AI-assisted debugging suggestions. - To evaluate how AI explains, detects, and fixes different types of coding errors. - To build confidence in using AI to perform structured debugging practices. Lab Outcomes (LOs): After completing this lab, students will be able to: - Use AI tools to detect and correct syntax, logic, and runtime errors. - Interpret AI-suggested bug fixes and explanations. - Apply systematic debugging strategies supported by AI-generated insights. Refactor buggy code using responsible and reliable programming patterns.		
	Task 1 (Mutable Default Argument – Function Bug) Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it. # Bug: Mutable default argument <pre>def add_item(item, items=[]): items.append(item) return items print(add_item(1)) print(add_item(2))</pre> Expected Output: Corrected function avoids shared list bug.		

	lab7.5-4054.py lab7.5-4054.py > add_item > items <pre>1 # Task 1 2 # Bug: Mutable default argument 3 def add_item(item, items=None): 4 if items is None: 5 items = [] 6 items.append(item) 7 return items 8 print(add_item(1)) 9 print(add_item(2))</pre> PS C:\Users\alish\OneD... e c:/Users/alish/OneD... [1] [1, 2] PS C:\Users\alish\OneD... e c:/Users/alish/OneD... [1] [2] PS C:\Users\alish\OneD...
	Task 2 (Floating-Point Precision Error) Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance. # Bug: Floating point precision issue <pre>def check_sum(): return (0.1 + 0.2) == 0.3 print(check_sum())</pre> Expected Output: Corrected function lab7.5-4054.py lab7.5-4054.py > check_sum <pre>12 # Task 2 13 # Bug: Floating point precision issue 14 def check_sum(): 15 return abs((0.1 + 0.2) - 0.3) < 1e-9 16 → return (0.1 + 0.2) == 0.3 17 print(check_sum()) 18</pre> e c:/Users/alish... • False PS C:\Users\alish... e c:/Users/alish... • True ○ PS C:\Users\alish...
	Task 3 (Recursion Error – Missing Base Case) Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix. # Bug: No base case <pre>def countdown(n): print(n) return countdown(n-1) countdown(5)</pre> Expected Output : Correct recursion with stopping condition.

	lab7.5-4054.py • lab7.5-4054.py > countdown <pre>19 # Task 3 20 # Bug: No base case 21 def countdown(n): 22 print(n) 23 if n <= 0: 24 return 25 return countdown(n-1) countdown(5)</pre> File "c:\Users\alish\OneDrive\Documents\lab7.5-4054.py": return countdown(n-1) ^^^^^^^^^^^^ [Previous line repeated 996 more times] RecursionError: maximum recursion depth exceeded PS C:\Users\alish\OneDrive\Documents> python lab7.5-4054.py 5 4 3 2 1
--	--

	Task 4 (Dictionary Key Error) Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it. # Bug: Accessing non-existing key <pre>def get_value(): data = {"a": 1, "b": 2} return data["c"] print(get_value())</pre> Expected Output: Corrected with .get() or error handling.
	lab7.5-4054.py • lab7.5-4054.py > get_value <pre>29 # Task 4 30 # Bug: Accessing non-existing key 31 def get_value(): 32 data = {"a": 1, "b": 2} 33 return data["c"] 34 return data.get("c", None) 35 print(get_value())</pre> return data["c"] ~~~~~ KeyError: 'c' PS C:\Users\alish\OneDrive\Documents> python lab7.5-4054.py None

	Task 5 (Infinite Loop – Wrong Condition) Task: Analyze given code where loop never ends. Use AI to detect and fix it. # Bug: Infinite loop <pre>def loop_example(): i = 0 while i < 5: print(i)</pre> Expected Output: Corrected loop increments i.
--	---

	lab7.5-4054.py lab7.5-4054.py > ... <pre>37 # Task 5 38 # Bug: Infinite loop 39 def loop_example(): 40 i = 0 41 while i < 5: 42 print(i) 43 i += 1 44 loop_example()</pre> PS C:\Users\alish\OneDrive\ e c:/Users/alish/OneDrive/D 0 1 2 3 4
	Task 6 (Unpacking Error – Wrong Variables) Task: Analyze given code where tuple unpacking fails. Use AI to fix it. # Bug: Wrong unpacking a, b = (1, 2, 3) Expected Output: Correct unpacking or using _ for extra values. lab7.5-4054.py X lab7.5-4054.py > ... <pre>48 # Task 6 49 # Bug: Wrong unpacking 50 a, *b = (1, 2, 3) 51 print(a) # 1 52 print(b) # [2, 3]</pre>
	Task 7 (Mixed Indentation – Tabs vs Spaces) Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it. # Bug: Mixed indentation <pre>def func(): x = 5 y = 10 return x+y</pre> Expected Output : Consistent indentation applied.

	lab7.5-4054.py X lab7.5-4054.py > func > y <pre>55 # Task 7 56 # Bug: Mixed indentation 57 def func(): 58 x = 5 59 y = 10 60 return x+y 61 print(func())</pre>
	Task 8 (Import Error – Wrong Module Usage) Task: Analyze given code with incorrect import. Use AI to fix. # Bug: Wrong import <pre>import maths print(maths.sqrt(16))</pre> Expected Output: Corrected to import math lab7.5-4054.py X lab7.5-4054.py <pre>64 # Task 8 65 # Bug: Wrong import 66 import math 67 print(math.sqrt(16))</pre>
	Task 9 (Unreachable Code – Return Inside Loop) Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it. # Bug: Early return inside loop <pre>def total(numbers): for n in numbers: return n print(total([1,2,3]))</pre> Expected Output: Corrected code accumulates sum and returns after loop.

```
lab7.5-4054.py •
lab7.5-4054.py > total
70 # Task 9
71 # Bug: Early return inside loop
72 def total(numbers):
73     total_sum = 0
74     for n in numbers:
75         return n
76     total_sum += n
77     print(total([1,2,3]))
78     return total_sum
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.

```
lab7.5-4054.py •
lab7.5-4054.py > calculate_area > |
80 # Task 10
81 # Bug: Using undefined variable
82 def calculate_area(length, width):
83     return length * width
84     length = 5
85     width = 10
86     print(calculate_area(length, width))
```

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

```
def add_values():
    return 5 + "10"
```

`print(add_values())`

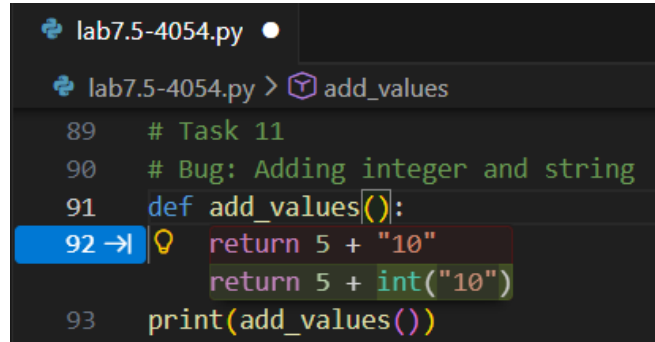
Requirements:

- Run the code to observe the error.
- AI should explain why `int + str` is invalid.
- Fix the code by type conversion (e.g., `int("10")` or `str(5)`).
- Verify with 3 assert cases.

Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.



```
lab7.5-4054.py
lab7.5-4054.py > add_values
89 # Task 11
90 # Bug: Adding integer and string
91 def add_values():
92 → return 5 + "10"
93     return 5 + int("10")
93 print(add_values())
```

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

`def combine():`

`return "Numbers: " + [1, 2, 3]`

`print(combine())`

Requirements:

- Run the code to observe the error.
- Explain why `str + list` is invalid.
- Fix using conversion (`str([1,2,3])` or `" ".join()`).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation

```
lab7.5-4054.py •
lab7.5-4054.py > combine

96 # Task 12
97 # Bug: Adding string and list
98 def combine():
99 → | return "Numbers: " + str([1, 2, 3])
100 print(combine())
```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

Bug: Multiplying string by float

```
def repeat_text():
    return "Hello" * 2.5
```

```
print(repeat_text())
```

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases.

```
lab7.5-4054.py 1 •
lab7.5-4054.py > repeat_text

103 # Task 13
104 # Bug: Multiplying string by float
105 def repeat_text():
106 → | return "Hello" * int(2.5)
107 print(repeat_text())
108
```

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

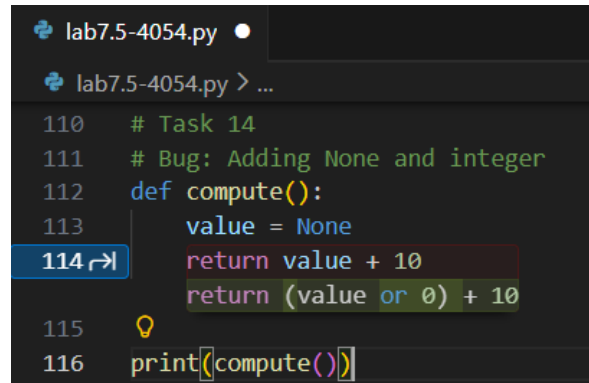
```
def compute():
    value = None
    return value + 10
```



```
print(compute())
```

Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.
- Validate using asserts.



```
lab7.5-4054.py
lab7.5-4054.py > ...
110 # Task 14
111 # Bug: Adding None and integer
112 def compute():
113     value = None
114     return value + 10
115     return (value or 0) + 10
116 print(compute())
```

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

```
def sum_two_numbers():
    a = input("Enter first number: ")
    b = input("Enter second number: ")
    return a + b
```

```
print(sum_two_numbers())
```

Requirements:

- Explain why input is always string.
- Fix using int() conversion.
- Verify with assert test cases.

lab7.5-4054.py

lab7.5-4054.py > ...

```
119 # Task 15
120 # Bug: Input remains string
121 def sum_two_numbers():
122     a = input("Enter first number: ")
123     b = input("Enter second number: ")
124     return a + b
125     return int(a) + int(b)
126 print(sum_two_numbers())
```